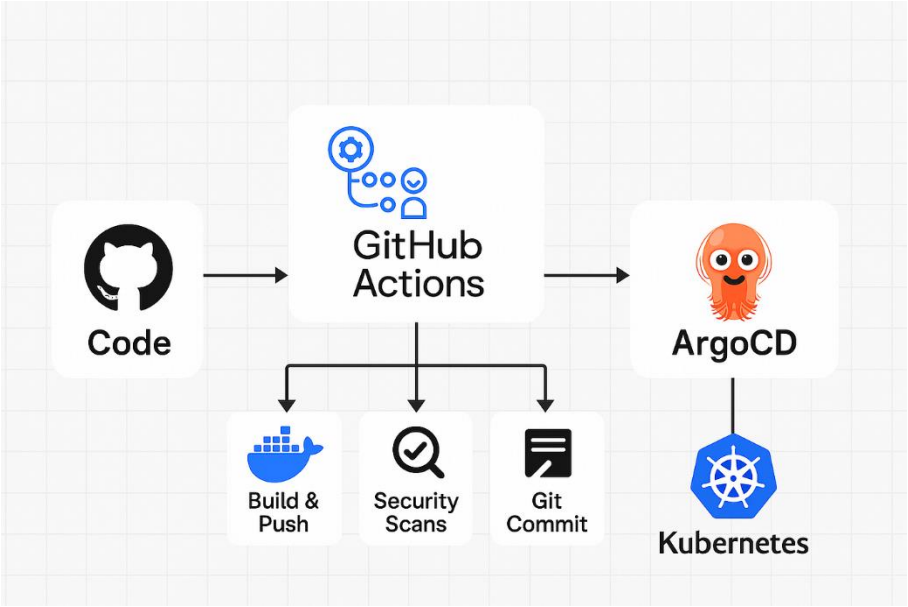
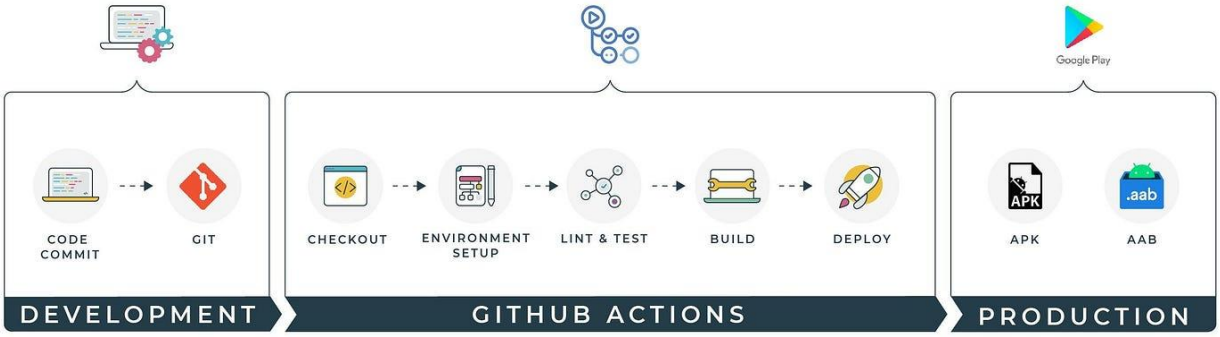


WEEK 10

CI/CD with GitHub Actions



What is CI/CD?

1.1 The Problem CI/CD Solves

Before CI/CD, software teams integrated and deployed code manually. Developers worked in isolation for days or weeks, then merged everything together only to discover dozens of conflicts and broken features. Deployments were manual, error-prone, and stressful events that sometimes took entire weekends.

CI/CD automates the entire process from code push to deployment. The moment a user pushes a commit to GitHub, an automated pipeline springs into action running tests, building the application, and verifying everything works without any human intervention.

1.2 Continuous Integration (CI)

Continuous Integration means integrating code changes frequently ideally every time a developer pushes to the repository and automatically verifying that the change does not break anything. A CI pipeline typically:

- Checks out the latest code from the repository
- Installs all dependencies
- Runs automated tests to verify correctness
- Builds the application (e.g. builds a Docker image)
- Reports success or failure back to the developer immediately

1.3 Continuous Delivery (CD)

Continuous Delivery extends CI by automatically preparing the application for deployment after every successful build. The code is always in a deployable state. In a full CD pipeline, the application might also be automatically deployed to a staging or production server. In this lab we focus on CI and the build stage of CD.

1.4 Key Terms

Term	Meaning
Pipeline	The full automated sequence of steps that runs on every code push
Workflow	A GitHub Actions pipeline: defined in a YAML file inside your repository
Job	A group of steps that run together on the same machine (runner)
Step	A single action inside a job, run a command, or use a pre-built action
Runner	The virtual machine that GitHub provides to execute your workflow
Trigger	The event that starts the workflow e.g. a push to main, or a pull request
Secret	An encrypted variable stored in GitHub for sensitive values like passwords and API keys

Artifact	A file produced by the pipeline (e.g. test reports, built binaries) that can be downloaded

GitHub Actions Fundamentals

2.1 How GitHub Actions Works

GitHub Actions is GitHub's built-in CI/CD platform. It is free for public repositories and has a generous free tier for private ones. Workflows are defined as YAML files stored inside your repository at a specific location: .github/workflows/. GitHub reads these files automatically and runs them when the trigger event occurs.

Every workflow file follows the same structure: a name, one or more trigger events, and one or more jobs. Each job contains a list of steps that run sequentially on a GitHub-hosted runner (a fresh Ubuntu, Windows, or macOS virtual machine).

2.2 Anatomy of a Workflow File

```
# .github/workflows/ci.yml

name: CI Pipeline              # Display name shown in GitHub Actions tab

on:                            # Triggers — when does this run?
  push:
    branches: [ main ]         # Run on every push to the main branch
  pull_request:
    branches: [ main ]         # Also run when a PR targets main

jobs:                          # One or more jobs to execute
  build-and-test:              # Job ID (used for referencing between jobs)
    runs-on: ubuntu-latest     # The type of runner to use

    steps:                     # Sequential list of steps in this job

      - name: Checkout code    # Step 1: get the repository code
        uses: actions/checkout@v4

      - name: Set up Python    # Step 2: install Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies   # Step 3: install packages
        run: pip install -r requirements.txt

      - name: Run tests        # Step 4: execute tests
        run: python -m pytest tests/ -v
```

2.3 Understanding the YAML Structure

Key	Purpose	Example Value
name	Human-readable name for the workflow	CI Pipeline
on	Events that trigger the workflow	push, pull_request
jobs	Collection of jobs to run	build-and-test
runs-on	The operating system for the runner	ubuntu-latest
steps	List of tasks inside a job	- name: ...
uses	Use a pre-built action from the GitHub marketplace	actions/checkout@v4
run	Execute a shell command directly	pip install flask
with	Pass parameters to an action	python-version: '3.11'
env	Set environment variables for a step or job	FLASK_ENV: testing
needs	Make a job wait for another job to finish first	needs: build-and-test

Writing Automated Tests for Flask

3.1 Why Tests are Essential in CI

A CI pipeline is only as useful as the tests it runs. Without tests, the pipeline just checks that the code can be installed not that it actually works correctly. Automated tests are the safety net that catches bugs before they reach users.

Python's pytest framework makes writing and running tests simple. Flask provides a built-in test client that lets you make HTTP requests to your app in tests without actually starting a server.

3.2 Creating Tests for the Flask CRUD API

Create a file called test_app.py inside your Flask project:

```
# test_app.py
import pytest
from app import app

@pytest.fixture
def client():
    app.config['TESTING'] = True
    with app.test_client() as client:
        yield client

# Test 1: Health check endpoint returns 200
def test_health_check(client):
    response = client.get('/api/health')
    assert response.status_code == 200
    data = response.get_json()
```

```
    assert data['status'] == 'ok'

# Test 2: Get all students returns a list
def test_get_students(client):
    response = client.get('/api/students')
    assert response.status_code == 200
    data = response.get_json()
    assert isinstance(data, list)

# Test 3: Get a student that does not exist returns 404
def test_get_student_not_found(client):
    response = client.get('/api/students/9999')
    assert response.status_code == 404

# Test 4: POST a new student with valid data returns 201
def test_add_student(client):
    new_student = {'name': 'Memoona', 'grade': 'A'}
    response = client.post('/api/students',
                           json=new_student)
    assert response.status_code == 201
    data = response.get_json()
    assert data['name'] == 'Memoona'

# Test 5: POST without required fields returns 400
def test_add_student_missing_field(client):
    response = client.post('/api/students',
                           json={'name': 'Sara'})
    assert response.status_code == 400
```

3.3 Running Tests Locally First

```
# Install pytest
$ pip3 install pytest

# Run all tests — always verify locally before pushing
$ python -m pytest test_app.py -v

# Expected output:
# test_app.py::test_health_check          PASSED
# test_app.py::test_get_students           PASSED
# test_app.py::test_get_student_not_found  PASSED
# test_app.py::test_add_student            PASSED
# test_app.py::test_add_student_missing_field PASSED
# 5 passed in 0.42s
```

Always run tests locally before pushing. If tests fail locally, fix them before they fail in the pipeline. A failing pipeline is visible to everyone on the team — it is much better to catch issues on your own machine first.

Building the CI Pipeline

4.1 Repository Setup

Your Flask project on GitHub (from Week 7) needs the following files before building the pipeline. Make sure all are committed and pushed:

- app.py: the Flask CRUD API with /api/health endpoint
- test_app.py: the test file from Section 3
- requirements.txt: must include flask, flask-cors, and pytest
- Dockerfile: from Week 9
- .dockerignore: excluding __pycache__, .env, *.pyc, *.log

4.2 Pipeline Stage 1: Lint and Test

Create the workflow directory and first pipeline file:

```
$ mkdir -p .github/workflows
$ nano .github/workflows/ci.yml
```

```
name: Flask CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    name: Run Tests
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest
```

```
      - name: Run pytest
        run: python -m pytest test_app.py -v

      - name: Verify health endpoint exists
        run: python -c "from app import app; assert '/api/health' in [r.rule
for r in app.url_map.iter_rules()]"
```

4.3 Pipeline Stage 2: Build Docker Image

Add a second job to the same workflow file that builds the Docker image after tests pass. The needs: test line ensures this job only runs if the test job succeeded.

```
  build-docker:
    name: Build Docker Image
    runs-on: ubuntu-latest
    needs: test              # Only run if the test job passed

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Build Docker image
        run: docker build -t flask-api:${{ github.sha }} .

      - name: Verify container starts correctly
        run: |
          docker run -d -p 5000:5000 --name test-container \
            flask-api:${{ github.sha }}
          sleep 3
          curl --fail http://localhost:5000/api/health
          docker stop test-container
          docker rm test-container
```

github.sha is a built-in GitHub Actions variable containing the unique commit hash. Using it as the image tag means every build is traceable back to the exact commit that produced it — a best practice for production deployments.

4.4 Pushing and Watching the Pipeline Run

```
# Stage all new files
$ git add .github/ test_app.py requirements.txt

# Commit with a clear message
$ git commit -m "Add CI pipeline with pytest and Docker build"

# Push to GitHub — this triggers the workflow immediately
$ git push origin main
```

After pushing, navigate to your GitHub repository in the browser and click the Actions tab. You will see your workflow running in real time. Click on it to watch each step execute, view its output, and see the green checkmark when it passes or a red X with the error message if it fails.

4.5 Using Secrets for Sensitive Values

Never put passwords, API keys, or tokens directly in your workflow YAML file, it is public and anyone can read it. GitHub Secrets store encrypted values that are injected into the pipeline at runtime.

1. Go to your GitHub repository → Settings → Secrets and variables → Actions
2. Click New repository secret
3. Name it (e.g. DOCKER_PASSWORD) and paste the value
4. Reference it in your workflow as: ${{ secrets.DOCKER_PASSWORD }}

```
# Example: using a secret in a workflow step
      - name: Log in to Docker Hub
        run: echo ${{ secrets.DOCKER_PASSWORD }} | docker login -u ${{
secrets.DOCKER_USERNAME }} --password-stdin
```

> Never echo or print a secret value in your workflow, even by accident. GitHub automatically masks known secret values in logs, but any transformation of the secret (e.g. base64 encoding) will not be masked.

Practice Exercise

Part A: Prepare the Repository

1. Open the GitHub repository you created in Week 7 for your Flask API
2. Clone it locally if needed and navigate into it

```
$ git clone https://github.com/your-username/flask-api.git
$ cd flask-api
```

3. Ensure requirements.txt includes all needed packages

```
$ cat requirements.txt
# Should contain: flask, flask-cors, pytest
```

4. Ensure app.py has the /api/health endpoint. If not, add it:

```
# Add inside app.py
@app.route('/api/health', methods=['GET'])
def health():
    return jsonify({'status': 'ok', 'message': 'Flask is running'}), 200
```

Part B: Write and Run Tests Locally

5. Create test_app.py with all 5 tests from Section 3.2

```
$ nano test_app.py
```

6. Install pytest and run tests locally — all 5 must pass before continuing

```
$ pip3 install pytest
$ python -m pytest test_app.py -v
```

Part C: Set Up the GitHub Actions Workflow

7. Create the workflows directory and CI file

```
$ mkdir -p .github/workflows
$ nano .github/workflows/ci.yml
```

8. Write the full two-job pipeline from Sections 4.2 and 4.3

9. Commit and push everything to trigger the pipeline

```
$ git add .
$ git commit -m "Add CI pipeline with tests and Docker build"
$ git push origin main
```

10. Open GitHub in the browser → Actions tab → watch both jobs run

11. Deliberately break a test to see the pipeline fail, then fix it

```
# Edit test_app.py — change a correct assertion to a wrong one
# e.g. change assert response.status_code == 200 to assert
response.status_code == 999
$ git add test_app.py
$ git commit -m "Intentional failing test"
$ git push origin main
# Watch the pipeline fail in the Actions tab
# Then revert the change, push again, and watch it go green
```

Lab Evaluation Tasks

Both tasks are demonstrated live on GitHub. Open your repository's Actions tab to verify pipeline runs, view logs, and check pass/fail status. Document your task as well, Each task carries equal marks.

Lab Task 1: Full CI Pipeline (Tests and Docker Build)

Build a complete two-job GitHub Actions pipeline for your Flask CRUD API. The pipeline must run automatically on every push to main and both jobs must show green (passing) in the Actions tab.

1. Ensure your GitHub repository contains: app.py (with /api/health), requirements.txt (flask, flask-cors, pytest), Dockerfile, and .dockerignore
2. Write test_app.py with at least 5 tests covering: health check (200), get all students (200 + list), get non-existent student (404), add valid student (201 + name check), add student missing a field (400)
3. Create .github/workflows/ci.yml with two jobs: 'test' (checkout, setup Python 3.11, install deps, run pytest) and 'build-docker' (needs: test, checkout, docker build, run container, curl health, stop container)
4. Push to GitHub and confirm both jobs complete successfully — show the instructor the green checkmarks in the Actions tab
5. Click on a completed run and show the instructor the step-by-step log output for the pytest step — all 5 tests must be visible as PASSED
6. Show the Docker build step log — the instructor will verify the image was built with the commit SHA as the tag

Lab Task 2: (Pipeline Resilience and Secrets)

Demonstrate that the CI pipeline correctly catches failures and that you understand how to use GitHub Secrets safely.

1. Deliberately introduce a bug in app.py change the /api/health route to return status code 500 instead of 200
2. Push the broken code to main and watch the pipeline run the test job must fail at the pytest step
3. Take a screenshot or note the exact error message GitHub shows in the failed step log
4. Fix the bug, push again, and verify the pipeline goes green
5. Add a third job to your workflow called 'report' that runs after build-docker and prints a summary: echo 'Pipeline complete for commit: ${{ github.sha }}' show the instructor this step in the Actions log
6. In your GitHub repository settings, create a dummy secret named LAB_SECRET with value yourname-devops-week10
7. Add a step to your workflow that uses this secret: echo 'Secret loaded successfully' (do NOT print the secret value itself) demonstrate the step runs without exposing the value in the logs
8. Write a short explanation (in a file called ci_notes.txt committed to the repo): what would happen to the build-docker job if the test job fails? Why is this important?